Architecture with Dynamic Associative Memory in Transformer Models

Alex Goldenstein

Status: Comprehensive Architectural and Methodological Specification

August 7, 2026

---

1 Introduction and Architectural Problem Statement

The proposed Cognitive Master Architecture is a hybrid cognitive architecture combining neural processing methods (Transformer) with external structured associative memory (LTM with graph topology and offline consolidation). Rather than treating conventional language models as collections of isolated token operations, this architecture introduces a structure oriented toward continuous accumulation and integration of new experience and knowledge. Previously acquired knowledge is used in subsequent communication, enabling progressively better results. The unit of communication is a dialogue, technically accumulating the context of a session.

Dialogue ($\mathcal{D}$)

Dialogue $\mathcal{D} = \{T_1, T_2, \ldots, T_N\}$ is formalized as a single dynamic process and an integral unit of discussion. Each turn T_n produces semantic activity that accumulates in the dialogue graph $G_{\mathcal{D}}$, recording transitions between topics and user intentions.

D-Context ($\mathbf{s}_{\mathcal{D}}$)

D-Context is the state of the current dialogue maintained inside the associative-memory module. It aggregates the semantic core of the discussion—topic, intentions, and contextual constraints—into the integrated state $\mathbf{s}_{\mathcal{D}}$. D-Context is created when a new dialogue starts. The first message T_1 initializes and anchors it; full memory information conditioned on D-Context begins to flow back into the Transformer from the second message T_2 onward.
The classical Transformer architecture exhibits a strict dichotomy between static parametric memory in the weight matrices ($W_Q, W_K, W_V, W_O, W_{\text{mlp}}$) and the dynamic but strictly length-limited Attention Context Window.
Adapting to new domains, tasks, or real-world facts requires either resource-intensive parametric adaptation (Fine-Tuning / PEFT), with the risk of catastrophic forgetting, or loading large datasets into the prompt (In-Context Learning / RAG), producing quadratic computational growth $\mathcal{O}(N^2)$ and noise.

Goal and Conceptual Solution

Construct a **hybrid multi-level semantic-associative memory system** functioning as an internal–external Transformer module:

1. **Level I (Base Invariant):** Extract and fix ("freeze") a universal asemantic geometric basis of the hidden activation space, $\mathcal{B} \subset \mathbb{R}^d$, once for the entire model.

2. **Level II (Dynamic Semantic Graph):** For a specific downstream task T, automatically prune irrelevant basis elements and impose directed causal and co-activation edges, forming the directed semantic graph G_T.

3. **Level III (OOD Induction):** As the external world changes, detect residual energy in activation space and inductively create new graph nodes and edges in real time without retraining the base neural network (Zero Re-Training / Continual Learning).

2 Closed-Loop Interaction between the Transformer and Associative Memory

Constructing the dictionary and semantic graph defines the structure of memory, but using that memory requires an explicit return path from memory to the Transformer. In the proposed architecture this path is implemented directly in the Transformer's internal hidden-state space.

Two-Phase Target-Layer Interface

Memory communication uses a target layer l_{target} of the selected Transformer model. The selected layer is simply doubled: the original part becomes the source for update, while the new part becomes the interface into which associative memory inserts its messages. This creates two logical directions of communication:

1. **Read Phase (Input Pole):** at the input to l_{target}, the current hidden state $h_t^{(l^{\text{in}}_{\text{target}})}$ is captured. It is used as a query to associative memory and to update D-Context without interrupting the Transformer's main computational stream.

2. **Return and Reintegration Phase (Output Pole):** associative memory uses the query, D-Context, and active structure G_T to form the latent memory vector Δh_t. It is returned to the Transformer at the output of the target layer and integrated into the Residual Stream:

$$\tilde{h}_t^{(l^{\text{out}}_{\text{target}})} = h_t^{(l^{\text{out}}_{\text{target}})} + \Delta h_t^{(\mathcal{D})}. \tag{1}$$

The resulting closed loop is

$$\text{Transformer hidden state} \rightarrow \text{Associative Memory} \rightarrow \text{D-Context / } G_T \rightarrow \Delta h_t \rightarrow \text{Transformer}.$$

Introducing an internal language elegantly solves the direct "vectors → graph" interface problem. In this interpretation the system becomes a Neuro-Symbolic AI hybrid in which the Transformer acts as the "processor" and the internal language as a "system assembler" issuing commands to memory and execution blocks.
In effect, the preceding discussion defines the language of internal communication of the cognitive system. It is fully determined by the selected Transformer and its original training. We now consider how the system's associative memory can learn this language and use it.

How the Internal Language Solves the Translation Problem

1. **Separation of responsibility:** the Transformer no longer needs to know exactly how the graph is organized in the database. Its task is to reason and express intent in a strictly deterministic machine-readable language (symbolic protocol).

The system's internal language is the Transformer's own internal language. It is read from an optimally selected internal layer. Associative memory knows nothing except this language: it learns it from scratch, first building activation patterns and then, on their basis, sequences and temporal relationships.
This conceptual shift changes the architecture completely, turning it from a neuro-symbolic hybrid into a purely endogenous neuro-associative system.

If the internal language consists of vector activations of one internal Transformer layer (Internal Latent Representation), while associative memory is initially "blind" to human language and learns from those activations from scratch, the system operates directly in the Transformer's internal representation space.

Architectural View

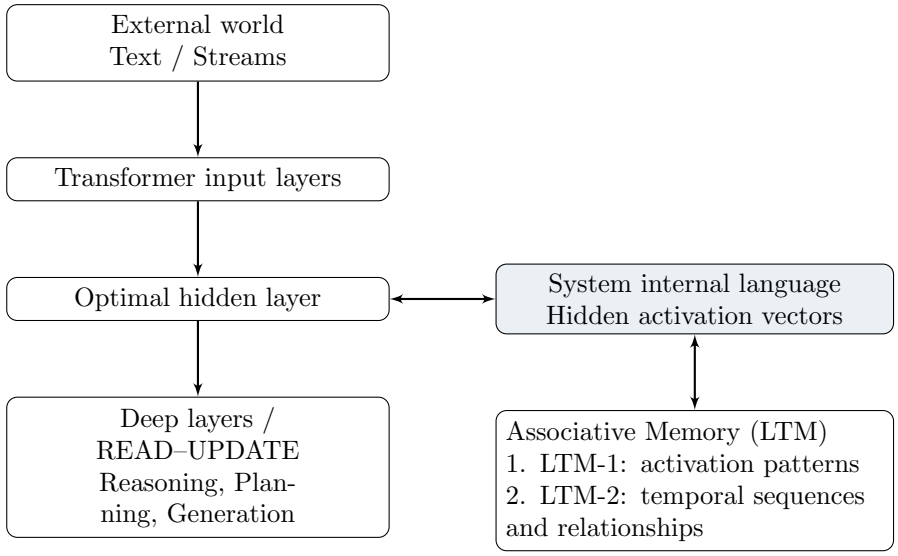


Figure 1: The Transformer's endogenous internal language as the associative-memory interface.

Main Advantages of the Concept

1. **Zero Translation Loss:** continuous meanings do not have to be translated into discrete JSON, Cypher, or a textual DSL. Memory operates on the native mathematical objects—vectors and tensors—used by the Transformer itself.

2. **Semantic Density and Unbiased Semantics:** the internal latent space of a deep Transformer layer contains richer semantics—context, intonation, and associations—than an individual human word.

3. **Self-Organization / Unsupervised Emergence:** LTM learns from scratch:

   - **Stage 1 (LTM-1):** memory records frequent static activation configurations, creating the base dictionary of the system's internal atomic meanings.
   - **Stage 2 (LTM-2):** using stable LTM-1 nodes, memory learns trajectories of state change over time, $\text{State}_t \rightarrow \text{State}_{t+1}$, forming episodic and causal memory.

New Fundamental Implementation Challenges

This approach solves the translator problem but introduces constraints.

1. Representation Drift / Concept Drift. The Transformer is a trainable and plastic system. If its weights are updated even slightly, the activation space of the selected hidden layer changes its coordinate system.
Risk: an activation vector V representing a particular concept before the neural-network update can point to another location afterward. Old LTM-1 and LTM-2 records then cease to correspond to the new internal-language coordinate system.
Solution: either freeze the selected layer rigidly (Frozen Internal Layer), or use space-alignment mechanisms (Vector Alignment / Procrustes Analysis / Projection Heads) to map old vectors into

the new coordinate system. **The optimal choice, however,** is to use a mature Transformer and provide all plasticity through associative-memory learning.

3. Defining What Counts as an Item in LTM-1. Because associative memory learns from scratch, it needs a criterion for deciding when the hidden-layer activation stream should create a new LTM-1 node and when it is merely a variation of an existing one.
Solution: online clustering based on Self-Organizing Maps (SOM), Predictive Coding, or Spiking Neural Networks (SNN) / Hebbian Learning. Memory creates a new LTM-1 node only when an incoming activation vector produces high Prediction Error relative to patterns already present in LTM-1.

Summary of this model. Using internal-layer activations as the native internal language makes the system fully endogenous. The Transformer acts as a perceptual front end, LTM-1 clusters stable activation patterns, and LTM-2 stores temporal chains and relationships among LTM-1 patterns. One technical option under consideration is Vector Quantization of the selected layer, converting the continuous latent stream into stable discrete entities suitable for LTM-1 and LTM-2 graph construction.

Double-Width Layer Scheme (Internal Memory Layer)

The Transformer is treated as a large language model with an additional input from read operations. This input enters the same internal layer from which update signals leave to train memory, and is integrated into answer generation by deeper layers. The layer producing update has double width; its second half receives read from memory, carrying information about the context of the current dialogue.

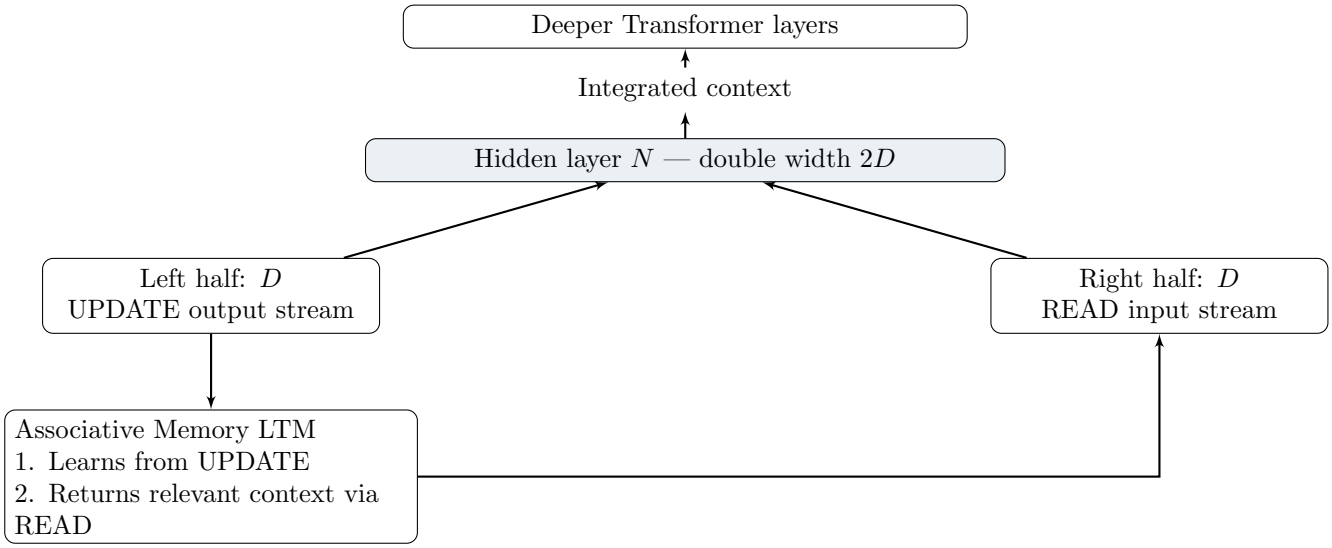


Figure 2: Double-width layer as a neural UPDATE/READ bus.

Architectural Advantages of the Scheme

1. **Closed-Loop Perception–Action:**
   - the left half expresses the Transformer's current internal state or intent (UPDATE), transmitting it to LTM in the internal language;
   - the right half receives the memory response (READ), containing associated patterns from LTM-1 and LTM-2;

- deeper Transformer layers combine both representations [UPDATE ‖ READ], integrating memory into reasoning and answer generation.

2. **No decoding delay:** memory does not need to convert the retrieved structure back into text. LTM accepts an UPDATE vector and returns the corresponding vector or mixture of vectors through READ. The Transformer receives the response in its native form.

3. **Synchronous interaction:** because UPDATE and READ share the same internal interface, memory acts not as an external database but as an extension of latent working space (Latent Coprocessor).

Key Challenges and Bottlenecks

2. Channel Balancing (Attention Masking / Gating). Problem: during early LTM training, the READ channel may return noise. If deep Transformer layers fully consume this channel, memory noise can disrupt the base model's linguistic coherence.
Solution: introduce a gating mechanism between channels:

$$\text{Layer}_{out} = W_1 \cdot \text{UPDATE} + \sigma(W_g \cdot [\text{UPDATE}, \text{READ}]) \odot (W_2 \cdot \text{READ}) \,. \tag{2}$$

The gate σ learns how strongly the deeper layers should trust READ depending on context.

3. Gradient Stability during Offline Learning. If LTM learns autonomously during a Sleep phase and restructures LTM-1/LTM-2, the distribution of signals entering READ changes. To keep deep Transformer layers from depending on uncontrolled distribution drift, READ should pass through LayerNorm with a fixed distribution.

Summary of Model Evolution

1. The LLM / Transformer acts as a universal cognitive processor.

2. Double-width layer N acts as a neural bus: the right half receives context from past episodes and the left half transmits the current state.

3. In offline Sleep, LTM-2 is compressed, unimportant details are removed, or entire episodes are forgotten, preventing uncontrolled growth and forming abstractions.

Dialogue Session Lifecycle

The first external request tells memory that a dialogue must begin. The next request activates the existing dialogue and, within it, forms a message for the Transformer. When a new request reaches the target Transformer layer, it generates an update that changes the dialogue context for subsequent requests and records the new state in memory.

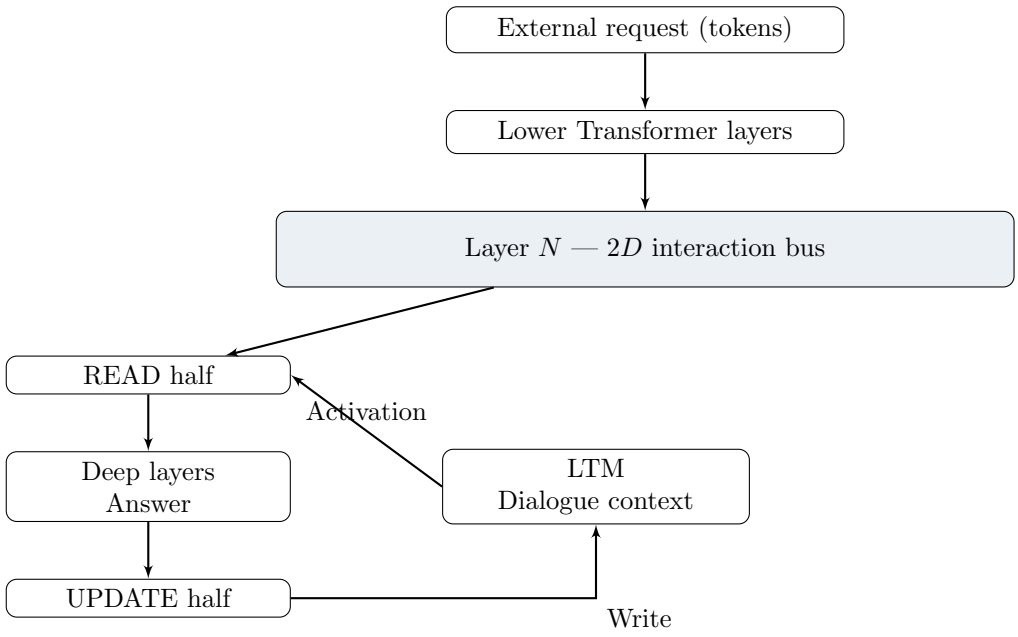


Figure 3: Stepwise READ/UPDATE cycle within an active dialogue.

Step 1: Initialization (first external request).

1. **Input:** an external request enters the system.
2. **Start signal:** the lower Transformer layers reach layer N; UPDATE generates a "Start dialogue" trigger vector.
3. **Associative-memory response:** LTM recognizes the trigger, creates a new session node/-context in LTM-2, and initializes READ.
4. **Result:** a pointer to the new dialogue is created and memory is ready to accumulate the episode.

Step 2: Active Phase (each subsequent request). 1. Reading existing context (READ Phase):

- a new user request enters the Transformer;
- on reaching layer N, the system takes the current dialogue-graph state from LTM-2 and feeds its latent representation into READ;
- deeper Transformer layers receive a memory vector compressing previous facts, roles, and dialogue context;
- the Transformer generates a response to the user.

2. Writing and updating context (UPDATE Phase):

- UPDATE sends LTM the revised internal state: what happened, which LTM-1 concepts were involved, and the new causal step;
- LTM consumes UPDATE and extends the current dialogue graph in LTM-2, linking the new step to previous ones.

Step 3: Consolidation (session end / transition to Sleep). When the dialogue stops, the accumulated LTM-2 session graph is marked as a Completed Logical Episode. During Sleep the episode graph is consolidated: association weights in LTM-1 are updated, while the LTM-2 dialogue graph is generalized and compressed into long-term experience.

Architectural Consequences

1. **The Transformer context window becomes practically "infinite":** the Transformer need not retain the entire previous text in the standard Attention window; it receives a compressed state of the current dialogue from LTM via READ.
2. **Memory as a State Machine:** LTM is not static storage but a dynamic neural state environment. Every UPDATE moves the dialogue graph from State_t to State_{t+1}.
3. **Dialogue locality in LTM-2:** each active dialogue forms a temporary branch or subgraph in LTM-2 connected to base LTM-1 elements.

Start-Signal Clarification. The start signal is the arrival of a signal at the Transformer's input layer. A new dialogue that has not yet received any update generates no read. This sets a strict initialization boundary condition: if the dialogue is empty, the system has nothing on which to perform associative search in LTM.

Two-Stage Cold-Start Protocol

To avoid using a dialogue context that has not yet been formed, interaction starts in two stages:

- **Turn T_1 — initialization.** The hidden state of the first message initializes and anchors D-Context. Feedback injection is disabled: $\Delta h_{T_1} = \mathbf{0}$.
- **Turn T_2+ — active interaction.** Beginning with the second message, associative memory uses accumulated D-Context to retrieve relevant information; returned state Δh_{T_n} is reintegrated into the Transformer Residual Stream.

End-to-End Reintegration

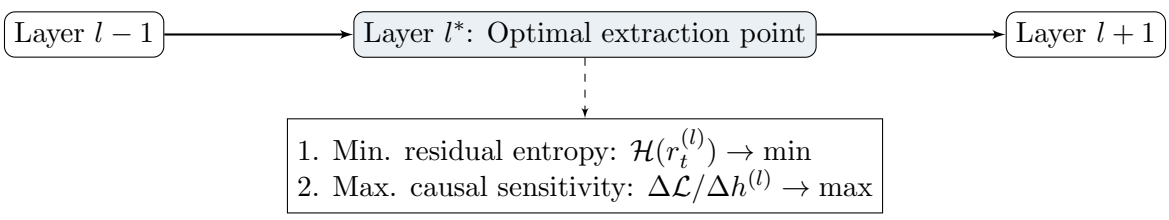
After the state is modified at $l^{\text{out}}_{\text{target}}$, subsequent Transformer layers operate normally. Over a small number of additional layers introduced together with the doubling of l_{target}, information returned by memory should smoothly merge into the common stream and participate in all subsequent computations up to output-logit formation:

$$h_t^{(l)} = h_t^{(l-1)} + \text{SelfAttention}^{(l)}\Big(\text{LN}(h_t^{(l-1)})\Big) + \text{MLP}^{(l)}\Big(\text{LN}(h_t^{(l-1)})\Big), \tag{3}$$

$$P(y_t \mid y_{<t}) = \text{Softmax}\Big(W_{\text{head}}\,\text{LN}(h_t^{(L)})\Big). \tag{4}$$

3 Automatic Layer Selection (Layer Entropy Profiling)

Hidden states $h_t^{(l)} \in \mathbb{R}^d$ change their functional nature radically with network depth $l \in [1, L]$. Early layers are saturated with surface syntactic noise, while the deepest layers exhibit representation degeneration before tokenizer projection.
To avoid heuristic layer selection, a **Layer Entropy Profiler** is used:



Mathematical Criterion

The optimal layer index l^* is defined as the global minimum of the differential entropy of residual-vector distributions subject to exceeding the task's causal-sensitivity threshold:

$$l^* = \arg\min_{l \in [1,L]} \mathcal{H}\Big(h^{(l)} - \mathbf{P}_{\mathcal{B}} h^{(l)}\Big) \quad \text{subject to} \quad \mathbb{E}\left[\left\| \frac{\partial \mathcal{L}_{\text{task}}}{\partial h^{(l)}} \right\|\right] > \tau_{\text{causal}}. \tag{5}$$

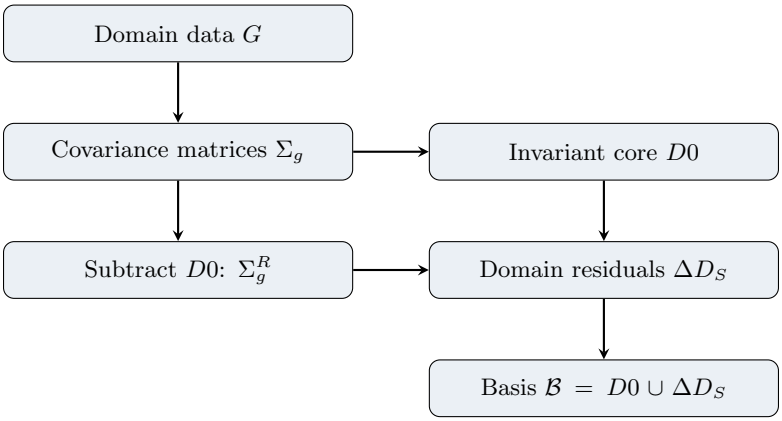
where $\mathcal{H}(\cdot)$ is differential entropy and $\mathbf{P}_{\mathcal{B}}$ is the projection operator onto the current basis.

One Possible Implementation of the Above

At the beginning of this document, we promised to **show how the system's associative memory can learn this language and use it**.
Everything below is one possible way of doing so. This section contains a great deal of mathematics and no new conceptual ideas; it can therefore be skipped without compromising the general understanding of the concept.
Most of the work described below is performed offline and requires only administrative access to an already trained Transformer model that will subsequently be used in the Cognitive system. The existing Transformer is used to generate the required statistics from datasets that will then be processed by the algorithms described below. For example, books passed through the Transformer can be grouped by genre and author, paintings by artist, and landscape photographs by landscape type, and so on.
The resulting large body of structured data will be used by the algorithms described below to determine the semantics of the internal language.

4 Stage I: Extracting the Asemantic Physical Basis $\mathcal{B}$

At the first stage, the system is isolated from notions of semantics and meaning. The statistical geometry of activation distributions is analyzed across a set of heterogeneous text domains $g \in G$.



Computation Algorithm

1. **Activation covariance collection:** at tap layer l^*, a covariance matrix is collected for each domain:

$$\Sigma_g = \frac{1}{N_g} \sum_{t=1}^{N_g} (h_t^g - \bar{h}^g)(h_t^g - \bar{h}^g)^T \in \mathbb{R}^{d \times d}. \tag{6}$$

2. **Central syntax invariant extraction ($D0$):** form $\Sigma_{\text{mean}} = \frac{1}{|G|} \sum_{g=1}^{|G|} \Sigma_g$. Spectral decomposition (SVD) extracts the first k_0 principal vectors, defining the orthonormal core $D0$.
3. **Specialized dictionary extraction (ΔD_S):** for each domain, compute a residual covariance matrix with the influence of $D0$ completely removed by orthogonal projection:

$$\Sigma_g^R = (I - P_{D0})\Sigma_g(I - P_{D0})^T, \qquad \text{where } P_{D0} = D0\, D0^T. \tag{7}$$

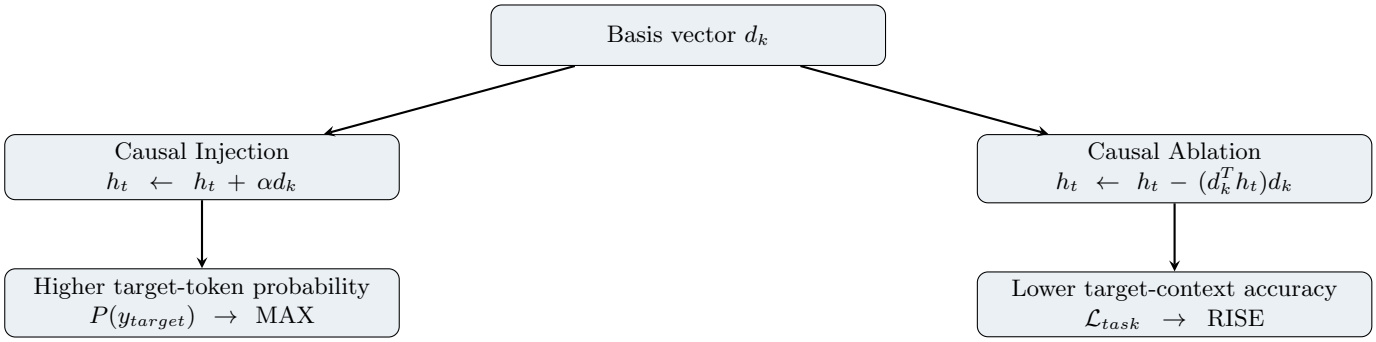
   The top-k_s vectors are selected from the SVD $\Sigma_g^R = U_g \Lambda_g U_g^T$.
4. **Frozen basis formation:**

$$\mathcal{B} = D0 \cup \left(\bigcup_{g \in G} \Delta D_{S,g} \right) = \{d_1, d_2, \ldots, d_M\} \subset \mathbb{R}^d. \tag{8}$$

   All vectors are normalized, $\|d_k\|_2 = 1$. This basis is fixed as the universal coordinate grid.

5 Stage II: Causal-Interventional Verification (Causal Patching)

Each basis vector $d_k \in \mathcal{B}$ is tested for functional efficacy by two opposite mathematical interventions in the neural network.



1. **Causal Injection:** in a neutral context at l^*, inject $h_t \leftarrow h_t + \alpha d_k$. The vector is considered causally active if the output-layer W_{unembed} logit shift produces a statistically significant probability increase for a specific token group.
2. **Causal Ablation:** in a context where the vector is naturally active, null it via $h_t \leftarrow h_t - (d_k^T h_t) d_k$. Retain it only if subtraction causally changes model behavior, $\Delta\mathcal{L}_{\text{task}} > \tau$.

6 Stage III: Task Reduction and Construction of Semantic Graph G_T

For a downstream task T, the model produces an activation sequence $H_T = \{h_1, h_2, \ldots, h_N\} \subset \mathbb{R}^d$. Each vector is decomposed over the frozen basis, $h_t = \sum_{k=1}^{M} c_{t,k} d_k + r_t$, where $c_{t,k} = h_t^T d_k$.

A. Semantic Filtering (Pruning)

The universal dictionary is compressed into the task-specific functional semantic dictionary $\mathcal{V}_T \subset \mathcal{B}$:

$$\mathcal{V}_T = \left\{ d_k \in \mathcal{B} \;\middle|\; \frac{1}{N} \sum_{t=1}^{N} |h_t^T d_k| > \tau_E \;\wedge\; \Delta\mathcal{L}_T(d_k) > \tau_L \right\}. \tag{9}$$

Vectors with low projection energy and zero causal influence are discarded.

B. Semantic Edge Induction

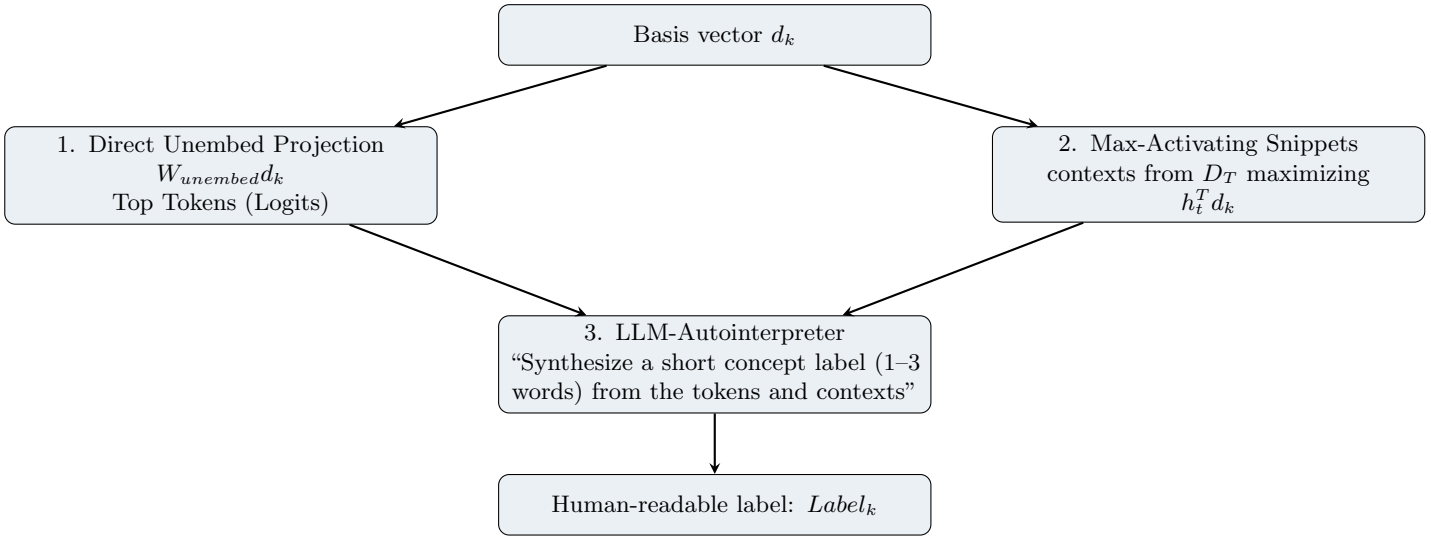
To turn $\mathcal{V}_T$ into a **Directed Semantic Graph** $G_T = (\mathcal{V}_T, E_T, W)$, two kinds of relationships are computed between nodes d_i and d_j:

1. **Synchronic Co-activation:**
$$W_{ij}^{\text{co}} = \frac{\sum_{t=1}^{N} (c_{t,i} - \bar{c}_i)(c_{t,j} - \bar{c}_j)}{\sqrt{\sum_t (c_{t,i} - \bar{c}_i)^2 \sum_t (c_{t,j} - \bar{c}_j)^2}}. \tag{10}$$
2. **Diachronic Causal Flow ($t \to t+1$):**
$$W_{i \to j}^{\text{causal}} = \frac{1}{N-1} \sum_{t=1}^{N-1} c_{t,i} c_{t+1,j}. \tag{11}$$

The final directed edge weight is $W_{ij} = \alpha W_{ij}^{\text{co}} + \beta W_{i \to j}^{\text{causal}}$.

7 Automatic Node Annotation (NodeAnnotator)

Abstract vectors are converted into a human-readable knowledge map by a three-stage annotation module:



1. **Direct Unembedding Projection:** top vocabulary tokens are obtained through $z_k = W_{\text{unembed}} d_k$.
2. **Max-Activating Dataset Snippets:** select training contexts producing the maximum projection amplitude $c_{t,k}$.
3. **LLM-Autointerpreter:** a lightweight interpreter generates strict JSON with a human-readable label (e.g. `Label: "Integral convergence condition"`).

8 Stage IV: Dynamic Memory Expansion under World Change (OOD Induction)

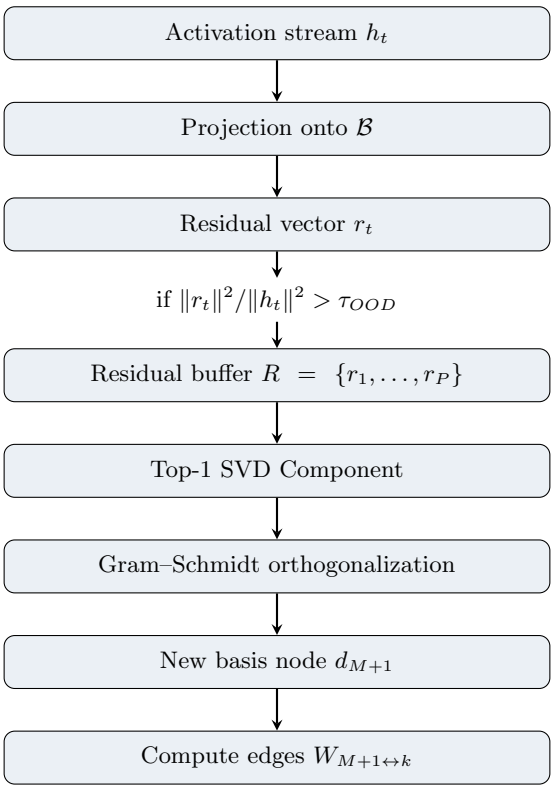
When fundamentally new real-world phenomena appear (Concept Drift), hidden states h_t can no longer be decomposed losslessly over the frozen basis $\mathcal{B}$.

1. Out-Of-Distribution (OOD) Detection

Monitor the critical residual-energy level:

$$r_t = h_t - \sum_{k=1}^{M} (h_t^T d_k) d_k \quad \Longrightarrow \quad \text{OOD criterion: } \frac{\|r_t\|_2^2}{\|h_t\|_2^2} > \tau_{\text{OOD}}. \tag{12}$$

Persistent residuals are placed in a ring buffer $R \in \mathbb{R}^{P \times d}$.



2. New Item and Relationship Induction Algorithm

1. **Geometry extraction:** from residual buffer R, extract $d_{\text{raw}} = \text{Top-1 SVD}(R)$.
2. **Gram–Schmidt orthogonalization:** subtract projections onto all existing basis vectors:

$$d_{\text{orth}} = d_{\text{raw}} - \sum_{k=1}^{M} (d_{\text{raw}}^T d_k) d_k, \qquad d_{M+1} = \frac{d_{\text{orth}}}{\|d_{\text{orth}}\|_2}. \tag{13}$$

3. **Basis expansion:** $\mathcal{B}_{\text{new}} = \mathcal{B} \cup \{d_{M+1}\}$.
4. **Graph G_T edge integration:** `DynamicEdgeIntegrator` computes synchronic and causal relationships between new node $M+1$ and active graph nodes from buffered contexts.

9 Software Implementation: `AssociativeMemoryEngine` Module

The Python code below demonstrates the complete OOD-detection, item-induction, and dynamic edge-construction pipeline:

```
import torch
import torch.nn as nn
import networkx as nx

class AssociativeMemoryEngine(nn.Module):
    def __init__(self, frozen_basis: torch.Tensor, ood_threshold: float =
        0.2,
                 buffer_capacity: int = 100, device: str = "cuda"):
        super().__init__()
        self.device = device
        self.ood_threshold = ood_threshold
        self.capacity = buffer_capacity
        # Normalized asemantic basis B [M, d]
        self.B = nn.Parameter(torch.nn.functional.normalize(frozen_basis, p
            =2, dim=1).to(device), requires_grad=False)
        self.residual_buffer = []

    @torch.no_grad()
```

```
    def process_step(self, h_t: torch.Tensor) -> tuple[torch.Tensor, torch.
        Tensor]:
        """Accept activation h_t [batch, d]; return C [batch, M] and R_t [
            batch, d]."""
        C = torch.matmul(h_t, self.B.T)
        h_reconstructed = torch.matmul(C, self.B)
        R_t = h_t - h_reconstructed
        energy_h = torch.norm(h_t, dim=-1) ** 2
        energy_r = torch.norm(R_t, dim=-1) ** 2
        ratio = energy_r / (energy_h + 1e-8)
        ood_mask = ratio > self.ood_threshold
        if torch.any(ood_mask):
            for r_vec in R_t[ood_mask]: self.residual_buffer.append(r_vec.
                cpu())
        return C, R_t

    @torch.no_grad()
    def try_induction_new_item(self, graph: nx.DiGraph, edge_threshold:
        float = 0.15) -> tuple[nx.DiGraph, int | None]:
        if len(self.residual_buffer) < self.capacity: return graph, None
        R_mat = torch.stack(self.residual_buffer[:self.capacity]).to(self.
            device)
        _, _, Vh = torch.linalg.svd(R_mat, full_matrices=False)
        d_raw = Vh[0]
        proj = torch.matmul(self.B, d_raw)
        d_orth = d_raw - torch.matmul(proj, self.B)
        norm_val = torch.norm(d_orth)
        if norm_val < 1e-5:
            self.residual_buffer.clear(); return graph, None
        d_new = (d_orth / norm_val).unsqueeze(0)
        new_node_id = self.B.shape[0]
        self.B = nn.Parameter(torch.cat([self.B.data, d_new], dim=0),
            requires_grad=False)
        active_nodes = list(graph.nodes())
        graph.add_node(new_node_id, label=f"OOD_Concept_{new_node_id}")
        if active_nodes:
            c_new = torch.matmul(R_mat, d_new.T).squeeze()
            active_basis = self.B[active_nodes]
            C_act = torch.matmul(R_mat, active_basis.T)
            c_new_c = c_new - c_new.mean()
            C_act_c = C_act - C_act.mean(dim=0, keepdim=True)
            cov = torch.matmul(c_new_c.unsqueeze(0), C_act_c).squeeze(0) / (
                self.capacity - 1)
            std_prod = (c_new.std() + 1e-8) * (C_act.std(dim=0) + 1e-8)
            W_corr = cov / std_prod
            W_np = W_corr.cpu().numpy()
            for i, target_node in enumerate(active_nodes):
                if abs(W_np[i]) > edge_threshold:
                    graph.add_edge(target_node, new_node_id, weight=float(
                        W_np[i]))
                    graph.add_edge(new_node_id, target_node, weight=float(
                        W_np[i]))
        self.residual_buffer.clear()
        return graph, new_node_id
```

Integration Example: `TransformerMemoryPipeline`

The following simplified module illustrates the complete state-read path, two-stage D-Context startup, and reintegration of latent memory state into subsequent Transformer layers:

```
class TransformerMemoryPipeline(nn.Module):
```

```
    def __init__(self, transformer_layers, frozen_basis, extract_idx,
        inject_idx, device="cuda"):
        super().__init__(); self.layers=transformer_layers; self.extract_idx
            =extract_idx; self.inject_idx=inject_idx
        self.B=nn.Parameter(torch.nn.functional.normalize(frozen_basis,p=2,
            dim=1).to(device),requires_grad=False)
        self.turn_counter=0; self.d_context_vector=torch.zeros(frozen_basis.
            shape[1],device=device)

    def start_new_dialogue(self):
        self.turn_counter=0; self.d_context_vector.zero_()

    def forward(self, hidden_states):
        self.turn_counter += 1; delta_h=None
        for idx, layer in enumerate(self.layers):
            # Input Pole: extract state
            if idx == self.extract_idx:
                h_extract=hidden_states.clone(); coeff=torch.matmul(
                    h_extract,self.B.T); h_proj=torch.matmul(coeff,self.B)
                if self.turn_counter == 1:
                    self.d_context_vector=h_proj.mean(dim=0); delta_h=torch.
                        zeros_like(hidden_states)
                else:
                    self.d_context_vector=0.85*self.d_context_vector+0.15*
                        h_proj.mean(dim=0)
                    delta_h=(h_proj*0.10)+(self.d_context_vector*0.05)
            hidden_states=layer(hidden_states)
            # Output Pole: reintegrate into the Residual Stream
            if idx == self.inject_idx and delta_h is not None: hidden_states
                =hidden_states+delta_h
        return hidden_states
```

The code above illustrates the interface rather than the final retrieval implementation: in the complete architecture, Δh_t must be formed by associative memory from the query, D-Context, and active graph G_T.

10 Two-Stage System Architecture

The architecture separates the construction of a stable physical basis from the task-dependent construction of semantic structure.

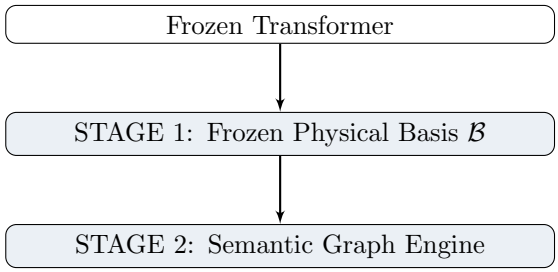


Figure 4: Two-stage system: a frozen physical basis and a dynamic Semantic Graph Engine.

Mathematical Apparatus of Stage 2

For a task T, let the sequence of hidden activations be

$$H_T = \{h_1, h_2, \ldots, h_N\}.$$

Each vector is decomposed over the frozen basis $\mathcal{B}$:

$$h_t = \sum_{k=1}^{M} c_{t,k} d_k + r_t, \qquad c_{t,k} = h_t^T d_k.$$

Step A. Semantic Filtering (Pruning & Sparsification)

A vector $d_k \in \mathcal{B}$ is considered an active semantic "word" for task T when its projection coefficient is not merely geometrically expressed but informationally relevant to solving the task.

1. **Activation Energy:** $E_T(d_k) = \frac{1}{N}\sum_{t=1}^{N} |h_t^T d_k|$.
2. **Task-Specific Ablation Sensitivity:** subtract direction d_k while solving T and measure the increase in task loss $\Delta\mathcal{L}_T(d_k)$.
3. **Masking filter:**
$$\mathcal{V}_T = \{d_k \in \mathcal{B} \mid E_T(d_k) > \tau_E \wedge \Delta\mathcal{L}_T(d_k) > \tau_L\}.$$
All vectors in $\mathcal{B} \setminus \mathcal{V}_T$ are removed from the computation for the specific task T.

Step B. Semantic Edges Induction

To convert $\mathcal{V}_T$ into a semantic graph $G_T = (\mathcal{V}_T, E, W)$, an adjacency matrix is computed between d_i and d_j using three edge types:

1. **Co-activation Edge / Synchronic:**
$$W_{ij}^{co} = \frac{\sum_{t=1}^{N}(c_{t,i} - \bar{c}_i)(c_{t,j} - \bar{c}_j)}{\sqrt{\sum_t (c_{t,i} - \bar{c}_i)^2 \sum_t (c_{t,j} - \bar{c}_j)^2}}.$$
2. **Causal Flow Edge / Diachronic:**
$$W_{i\to j}^{causal} = \frac{1}{N-\delta}\sum_{t=1}^{N-\delta} c_{t,i}\frac{\partial c_{t+\delta,j}}{\partial c_{t,i}}.$$
3. **Attention-Mediated Edge:**
$$W_{i\to j}^{attn} = \sum_{t_1,t_2} A_{t_1,t_2}^{(l)}(h_{t_1}^T d_i)(h_{t_2}^T d_j).$$

The final semantic edge has weight
$$W_{ij} = \alpha W_{ij}^{co} + \beta W_{i\to j}^{causal} + \gamma W_{i\to j}^{attn}.$$

Why This Scheme Is More Efficient

1. **Computational economy (Zero Re-Training):** SVD, SAE, or Diffusion Maps need not be repeated for every new task. The geometric physical basis $\mathcal{B}$ is computed once at Stage I.
2. **Task-space compression:** instead of analyzing the full $12\,288 \times 12\,288$ space, a task uses roughly $M_{\text{active}} \approx 50 \ldots 200$ key elements.
3. **Interpretability and graph coprocessing:** nodes $\mathcal{V}_T$ represent concepts used by the model and edges represent directed logical transitions.
4. **Graph Editing:** nodes and edges of G_T can be programmatically suppressed by acting on coefficients $c_{t,k}$, without fine-tuning the general-language basis.

11 Automatic Annotation of Semantic-Graph Nodes

Assigning text labels to vectors of the asemantic basis $\mathcal{B}$ transforms geometric graph G_T into a human-readable knowledge map. An individual $d_k \in \mathcal{B}$ may correspond not to a single word but to an abstract concept, syntactic role, or complex contextual polysemantic pattern.

Three-Stage Annotation Architecture

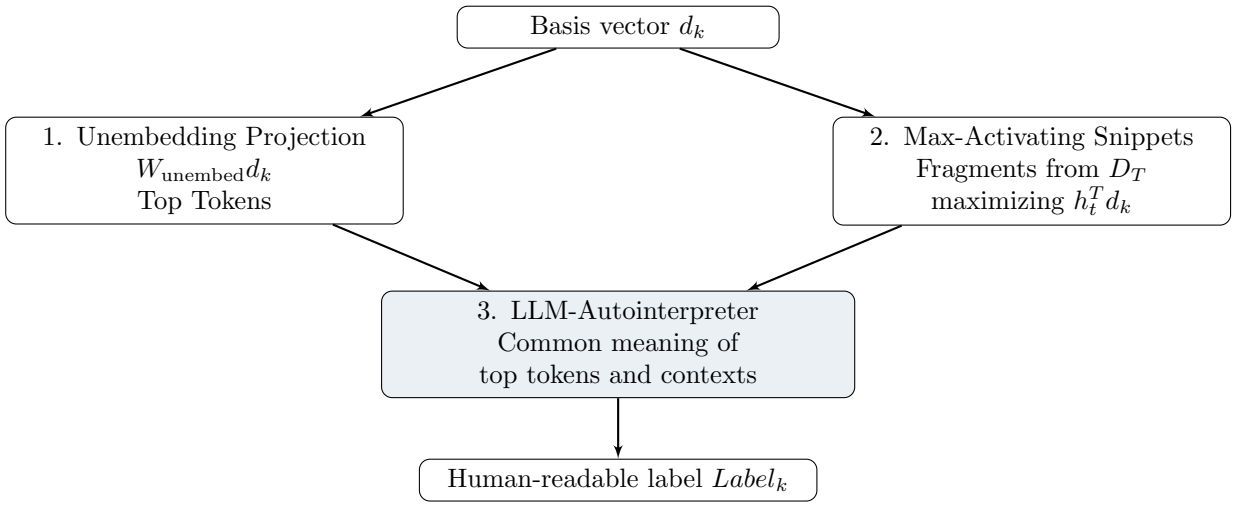


Figure 5: Three-stage automatic annotation of semantic-graph nodes.

Detailed Stages

Stage 1. Projection onto the Vocabulary Unembedding Matrix. Project d_k onto $W_{\text{unembed}} \in \mathbb{R}^{|V| \times d}$:

$$z_k = W_{\text{unembed}} d_k \in \mathbb{R}^{|V|}.$$

Select Top-K tokens with the largest logits z_k, yielding tokens, suffixes, or subwords directly activated by the vector at model output.

Stage 2. Max-Activating Dataset Snippets. Run corpus D_T through the model and retain Top-N text fragments maximizing $h_t^T d_k$. The context window around each position reveals abstractions that may not correspond to a single vocabulary token.

Stage 3. LLM-Autointerpretation. A compact annotation model receives top tokens and several maximally activating contexts and returns a structured label:

- `label`: short concept name;
- `category`: "Concept", "Syntax", "Style", or "Operator";
- `confidence`: confidence estimate.

Visual Result of Annotated Graph G_T

After the pipeline, the anonymous vector graph becomes readable:

- Node #412 → "Integral computation" (top tokens: $\int$, integral, dx);
- Node #891 → "Convergence condition" (top tokens: limit, bounded, converges);
- Edge #412 → #891 with weight 0.84 means that when the model addresses integrals it causally activates convergence checking.

Annotation is performed on demand only for the M_{active} nodes surviving task-T filtering, reducing LLM-interpreter calls from tens of thousands of basis elements to a few dozen active G_T nodes.

12 OOD Detection and Dynamic Dictionary Expansion

New real-world phenomena, terminology, or physical laws (Out-Of-Distribution / Concept Drift) can make the current activation state $h_t \in \mathbb{R}^d$ impossible to represent losslessly over frozen basis $\mathcal{B}$. A new object manifests as increased residual r_t.

Detection Phase

$$h_t = \sum_{k=1}^{M} c_{t,k} d_k + r_t, \qquad c_{t,k} = h_t^T d_k,$$

$$r_t = h_t - \sum_{k=1}^{M} (h_t^T d_k) d_k.$$

New-Item detection criteria:

1. **High Residual Energy:** $\frac{\|r_t\|_2^2}{\|h_t\|_2^2} > \tau_{\text{OOD}}$, with $\tau_{\text{OOD}} \approx 0.15 \ldots 0.25$.
2. **Persistent Residual Cluster:** a single outlier is treated as noise. Systematic residual accumulation into a dense cluster in $\mathbb{R}^d$ indicates a new semantic concept.

Ingestion Phase

When a stable cluster $R = \{r_1, r_2, \ldots, r_P\}$ is detected, Online Incremental Induction begins.

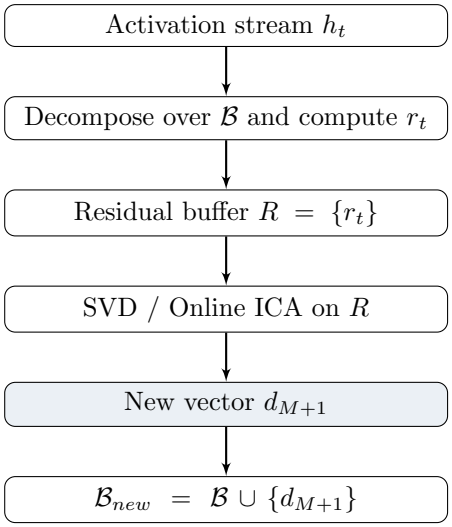


Figure 6: Online induction of a new dictionary element from persistent residuals.

Step A. Residual Buffer. All r_t exceeding τ_{OOD} are placed in a ring buffer $R \in \mathbb{R}^{P \times d}$.

Step B. Extract the New-Concept Direction. After accumulating P vectors (e.g. $P = 100 \ldots 500$), use $d_{\text{new}} = \text{Top-1 SVD}(R)$ or the normalized mean of stable residuals.

Step C. Gram–Schmidt Orthogonalization.

$$d_{\text{new}}^{orth} = d_{\text{new}} - \sum_{k=1}^{M} (d_{\text{new}}^T d_k) d_k, \qquad d_{M+1} = \frac{d_{\text{new}}^{orth}}{\|d_{\text{new}}^{orth}\|}.$$

Step D. Integration.

1. add d_{M+1} to $\mathcal{B}$: $\mathcal{B}_{updated} = \mathcal{B} \cup \{d_{M+1}\}$;
2. NodeAnnotator generates a label from contexts that produced r_t;
3. create node $M + 1$ in G_T and induce its semantic edges.

Architectural Advantages

1. **Continual Zero-Forgetting Learning:** new data do not retrain the Transformer or alter existing $d_1, \ldots, d_M$; a new orthogonal direction d_{M+1} is added.
2. **Noise vs. new-concept separation:** random fluctuations cancel in the buffer SVD while persistent geometric shifts survive.
3. **Knowledge-evolution audit:** every new item may receive an induction timestamp, allowing the system's world model to be tracked over time.

13 Integrating a New Item into the Semantic Graph

A new vector d_{M+1} in basis $\mathcal{B}$ is only the physical registration of a novel phenomenon. To make it a full element of model reasoning, semantic edges must be integrated into current graph G_T.

Dynamic Edge Induction & Integration Architecture

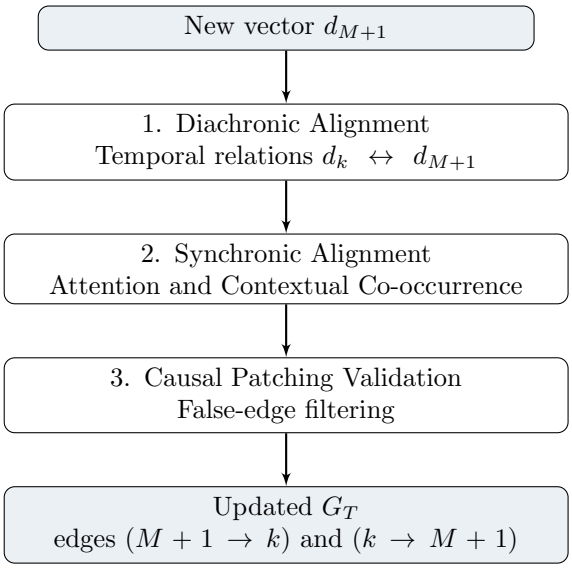


Figure 7: Integration of a new Item into the dynamic semantic graph.

Computing Semantic Relationships for d_{M+1}

After forming a new vector from residual buffer $R = \{r_1, r_2, \ldots, r_P\}$, retain activation sequences in which the concept appeared. To connect node $M+1$ to each existing active node $k \in G_T$, compute:

1. Forward/Backward Diachronic Flow. Incoming edge:

$$W^{causal}_{k \to M+1} = \frac{1}{P-1} \sum_{t=1}^{P-1} (h_t^T d_k)(h_{t+1}^T d_{M+1}).$$

Outgoing edge:

$$W^{causal}_{M+1 \to k} = \frac{1}{P-1} \sum_{t=1}^{P-1} (h_t^T d_{M+1})(h_{t+1}^T d_k).$$

2. Synchronic Context Coupling.

$$W^{co}_{M+1,k} = \frac{\mathrm{Cov}(h_t^T d_{M+1}, h_t^T d_k)}{\sigma(d_{M+1})\sigma(d_k)}.$$

3. Causal Patching for False-Edge Filtering. For the highest-weight candidate edges, perform a causal test:

1. in a context where d_k activates, artificially ablate it;
2. if activation of d_{M+1} on the next step falls by $\Delta > \tau_{\text{causal}}$, accept edge $e(k \to M+1)$ as causally valid.

Complete Lifecycle Example for a New Item

If a new vulnerability such as "DeepSeek-R1 Prompt Injection" appears in the external world, its lifecycle is:

1. **Detection:** $\mathcal{B}$ cannot fully represent the specific context, so residual r_t grows.
2. **Induction:** residuals accumulate and SVD extracts orthogonal vector d_{M+1}.
3. **Annotation:** NodeAnnotator assigns a human-readable label to node $M+1$.
4. **Semantic Integration:** DynamicEdgeIntegrator computes relationships with existing G_T nodes, for example an incoming edge from the general concept Prompt Injection and an outgoing edge to System Prompt Leak.

Thus G_T acquires structured knowledge about a new real-world object—its name, geometric vector, and place in the Transformer's causal reasoning chain—without retraining the base network.

14 Resolution of Key Limitations and Overall Result

The proposed architecture addresses the main theoretical vulnerabilities through the following built-in mechanisms:

Original vulnerability	**Architectural solution**
Nonlinear manifolds	Complex nonlinear concepts are encoded not by isolated vectors but by the **nonlinear topology of subgraph** G_T. Vectors d_k serve only as a local linear coordinate grid.
Layer sensitivity	Manual layer assignment is eliminated. Extraction point l^* is selected automatically by **Layer Entropy Profiling** at minimum residual entropy $\mathcal{H}(r^{(l)})$.
Capacity of $\mathbb{R}^d$ under OOD	Orthogonality is maintained by Gram–Schmidt. When linear independence is exhausted, the system moves to Overcomplete Dictionaries and sparse coding.
Causal validity	False correlations are filtered with two-pass **Causal Patching** (Injection / Ablation) during node and edge pruning.

15 Answers to Key Problems and Concluding Analysis

The architecture neutralizes the principal theoretical vulnerabilities through the following built-in mechanisms:

Initial vulnerability	**Solution within the system architecture**
Nonlinearity problem (Non-linear Manifolds)	Complex nonlinear concepts are encoded not by individual vectors but by nonlinear graph-subgraph topology G_T. Vectors d_k provide only a local linear coordinate grid.
Layer-selection sensitivity	Manual layer fixing is rejected. Tap layer l^* is selected automatically through Layer Entropy Profiling by minimizing residual entropy $\mathcal{H}(r^{(l)})$.
Capacity of $\mathbb{R}^d$ under OOD	Orthogonality is strictly maintained by Gram–Schmidt. Once linear independence is exhausted, the system moves to Overcomplete Dictionaries and sparse coding.
Causal validity	False correlations are removed through two-pass Causal Patching (Injection / Ablation) during node and edge filtering.

Final Conclusion

The resulting architecture is a framework for building interpretable Transformer models with continual learning (Continual Zero-Forgetting Learning). Translating the neural network's internal processes into the language of a geometric basis and semantic graphs G_T makes it possible to edit model knowledge, visualize decision logic, and dynamically integrate new real-world facts without costly and dangerous retraining of the base weights.